Word2Vec: Skip-Gram with Negative Sampling

Christian Kobriger

February 25, 2026

Contents

1 Setup **2**

2 The Softmax Formulation **2**
2.1 Why This Is Computationally Expensive 2

3 Hierarchical Softmax **2**

4 Negative Sampling **2**
4.1 Core Idea 2
4.2 The Negative Sampling Loss 3
4.3 The Noise Distribution 3

5 Gradient Derivations **3**
5.1 Sigmoid Derivative 3
5.2 A Useful Derivation 4
5.3 Gradient w.r.t. the Center Embedding $\mathbf{v}_c$ 4
5.4 Gradient w.r.t. the Positive Context Embedding $\mathbf{u}_o$ 4
5.5 Gradient w.r.t. a Negative Context Embedding $\mathbf{u}_{n_i}$ 5

6 Unified Vectorized Form **5**

1 Setup

The model maintains **two embedding matrices**:

- $\mathbf{V} \in \mathbb{R}^{|V| \times d}$: center word embeddings, with rows $\mathbf{v}_c$ for word c
- $\mathbf{U} \in \mathbb{R}^{|V| \times d}$: context word embeddings, with rows $\mathbf{u}_w$ for word w

2 The Softmax Formulation

The classical formulation defines the conditional probability using a softmax over the entire vocabulary:

$$P(o \mid c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \tag{1}$$

The training objective is to minimize the negative log-likelihood:

$$\mathcal{L}_{\text{softmax}} = -\log P(o \mid c) = -\mathbf{u}_o^\top \mathbf{v}_c + \log \sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c) \tag{2}$$

2.1 Why This Is Computationally Expensive

The denominator in Equation (1) requires summing over the **entire vocabulary** $|V|$ for every single training example. With vocabularies of $|V| \approx 10^5$–10^6 words, this means:

1. Computing $|V|$ dot products per training sample for the forward pass.
2. The gradient of the normalizing constant requires updating **all** context embeddings:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = -\mathbf{u}_o + \sum_{w \in V} P(w \mid c)\, \mathbf{u}_w \tag{3}$$

This is an expectation over the full vocabulary — an $O(|V| \cdot d)$ operation per training step.

For a corpus of billions of words, this is prohibitively expensive. Two key approximations address this: hierarchical softmax and negative sampling.

3 Hierarchical Softmax

Hierarchical softmax replaces the flat softmax with a **binary tree** over the vocabulary. This reduces the cost from $O(|V|)$ to $O(\log |V|)$ per training sample. However, negative sampling offers a simpler formulation and is frequently used in practice.

4 Negative Sampling

4.1 Core Idea

Instead of normalizing over the full vocabulary, negative sampling reformulates the problem as **binary classification**: distinguish real context pairs from randomly sampled "noise" pairs.

For a center word c, a true context word o, and k negative samples $n_1, \ldots, n_k$ drawn from a noise distribution $P_n(w)$:

- The positive pair (c, o) should have label $y = 1$

- Each negative pair (c, n_i) should have label $y = 0$

The probability that a pair is a real context pair is modeled as:

$$P(\text{real} \mid c, w) = \sigma(\mathbf{u}_w^\top \mathbf{v}_c) = \frac{1}{1 + \exp(-\mathbf{u}_w^\top \mathbf{v}_c)} \tag{4}$$

4.2 The Negative Sampling Loss

We maximize the log-probability that the positive pair is real and the negative pairs are fake:

$$\mathcal{L}_{\text{NEG}} = \log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{n_i}^\top \mathbf{v}_c) \tag{5}$$

Equivalently, we **minimize** the loss:

$$\boxed{\mathcal{L} = -\log \sigma(s^+) - \sum_{i=1}^{k} \log \sigma(-s_i^-)} \tag{6}$$

where $s^+ = \mathbf{v}_c^\top \mathbf{u}_o$ is the score for the positive pair and $s_i^- = \mathbf{v}_c^\top \mathbf{u}_{n_i}$ is the score for the i-th negative pair.

Note on the sign inside $\sigma(-s_i^-)$: We want $P(\text{fake} \mid c, n_i) \to 1$, i.e., $1 - \sigma(s_i^-) \to 1$. Using the identity $1 - \sigma(x) = \sigma(-x)$, this becomes $\sigma(-s_i^-)$, which provides the clean formulation in Equation (6).

4.3 The Noise Distribution

Negative samples are drawn from the smoothed unigram distribution:

$$P_n(w) = \frac{\text{count}(w)^{3/4}}{\sum_{w'} \text{count}(w')^{3/4}} \tag{7}$$

The 3/4 exponent smooths the distribution: compared to the raw unigram, it reduces the dominance of very frequent words (like "the") while giving rare words a higher sampling probability. This has been shown to work well empiraclly in [3]

5 Gradient Derivations

We now derive the gradients of the loss in Equation (6) with respect to each parameter. We will need the following property of the sigmoid function throughout.

5.1 Sigmoid Derivative

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \Longrightarrow \qquad \sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big) \tag{8}$$

We also use the identity:

$$1 - \sigma(x) = \sigma(-x) \tag{9}$$

5.2 A Useful Derivation

Since the same chain rule pattern appears in every gradient below, we derive it once.

For any scalar s:

$$\frac{\partial}{\partial s}\big[-\log\sigma(s)\big] = \sigma(s) - 1 \tag{10}$$

$$\frac{\partial}{\partial s}\big[-\log\sigma(-s)\big] = \sigma(s) \tag{11}$$

Proof of (10).

$$\frac{\partial}{\partial s}\big[-\log\sigma(s)\big] = -\frac{1}{\sigma(s)}\cdot\sigma'(s) = -\frac{\sigma(s)(1-\sigma(s))}{\sigma(s)} = -(1-\sigma(s)) = \sigma(s) - 1 \tag{12}$$

Proof of (11).

$$\begin{aligned}\frac{\partial}{\partial s}\big[-\log\sigma(-s)\big] &= -\frac{1}{\sigma(-s)}\cdot\sigma'(-s)\cdot(-1) = \frac{1-\sigma(-s)}{\cancel{\sigma(-s)}}\cdot\cancel{\sigma(-s)} \\ &= 1-\sigma(-s) = \sigma(s)\end{aligned} \tag{13}$$

where we used Equations (8) and (9). □

5.3 Gradient w.r.t. the Center Embedding $\mathbf{v}_c$

The loss is:

$$\mathcal{L} = \underbrace{-\log\sigma(s^+)}_{\text{positive term}} \underbrace{-\sum_{i=1}^{k}\log\sigma(-s_i^-)}_{\text{negative terms}}$$

Since $s^+ = \mathbf{v}_c^\top\mathbf{u}_o$ and $s_i^- = \mathbf{v}_c^\top\mathbf{u}_{n_i}$, we have $\frac{\partial s^+}{\partial\mathbf{v}_c} = \mathbf{u}_o$ and $\frac{\partial s_i^-}{\partial\mathbf{v}_c} = \mathbf{u}_{n_i}$.

Applying the chain rule with the lemma:

Positive term:

$$\frac{\partial}{\partial\mathbf{v}_c}\big[-\log\sigma(s^+)\big] = \underbrace{\big(\sigma(s^+)-1\big)}_{\text{from (10)}}\cdot\underbrace{\mathbf{u}_o}_{\partial s^+/\partial\mathbf{v}_c} \tag{14}$$

Negative terms:

$$\frac{\partial}{\partial\mathbf{v}_c}\big[-\log\sigma(-s_i^-)\big] = \underbrace{\sigma(s_i^-)}_{\text{from (11)}}\cdot\underbrace{\mathbf{u}_{n_i}}_{\partial s_i^-/\partial\mathbf{v}_c} \tag{15}$$

Combined:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{v}_c} = \frac{\partial\mathcal{L}}{\partial s^+}\cdot\frac{\partial s^+}{\partial\mathbf{v}_c} + \sum_{i=1}^{k}\frac{\partial\mathcal{L}}{\partial s_i^-}\cdot\frac{\partial s_i^-}{\partial\mathbf{v}_c} = \big(\sigma(s^+)-1\big)\,\mathbf{u}_o + \sum_{i=1}^{k}\sigma(s_i^-)\,\mathbf{u}_{n_i} \tag{16}$$

5.4 Gradient w.r.t. the Positive Context Embedding $\mathbf{u}_o$

Only the positive term depends on $\mathbf{u}_o$. Since $\frac{\partial s^+}{\partial\mathbf{u}_o} = \mathbf{v}_c$:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{u}_o} = \frac{\partial\mathcal{L}}{\partial s^+}\cdot\frac{\partial s^+}{\partial\mathbf{u}_o} = \big(\sigma(s^+)-1\big)\,\mathbf{v}_c \tag{17}$$

5.5 Gradient w.r.t. a Negative Context Embedding $\mathbf{u}_{n_i}$

Only the i-th negative term depends on $\mathbf{u}_{n_i}$. Since $\frac{\partial s_i^-}{\partial \mathbf{u}_{n_i}} = \mathbf{v}_c$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{n_i}} = \frac{\partial \mathcal{L}}{\partial s_i^-} \cdot \frac{\partial s_i^-}{\partial \mathbf{u}_{n_i}} = \sigma(s_i^-)\,\mathbf{v}_c \tag{18}$$

6 Unified Vectorized Form

The three gradient equations (16)–(18) share a common structure. Define:

- $\mathbf{u}_j$ for $j = 0, 1, \ldots, k$, where $j = 0$ is the positive context and $j \geq 1$ are negatives
- $s_j = \mathbf{v}_c^\top \mathbf{u}_j$ for all j
- $y_j = 1$ if $j = 0$ (positive), $y_j = 0$ if $j \geq 1$ (negative)

Then the gradient coefficient for each index j is:

$$\alpha_j = \sigma(s_j) - y_j \tag{19}$$

And all gradients take the form:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = \sum_{j=0}^{k} \alpha_j\,\mathbf{u}_j \tag{20}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_j} = \alpha_j\,\mathbf{v}_c \qquad \text{for each } j = 0, 1, \ldots, k \tag{21}$$

This is the form used in the implementation, where `grad_coeff` is the vector $[\alpha_0, \alpha_1, \ldots, \alpha_k]$.

References

[1] Yoav Goldberg and Omer Levy. word2vec explained: deriving mikolov et al.'s negative-sampling word-embedding method, 2014.

[2] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013.

[3] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality, 2013.